Savitribai Phule Pune University
Second Year of Computer Engineering & Computer Science and Engineering
(2024 Course)

PCC-254-COM : Database Management Laboratory

BY- Prof. Monika G. Ghodekar
AISSMSCOE, Assistant Professor

Assignment no 2

Write and execute SQL Data Definition Language(DDL) commands such as CREATE, ALTER, DROP, RENAME, and TRUNCATE to define and modify tables. Insert data into the tables and apply appropriate integrity constraints such as NOT NULL,UNIQUE,PRIMARY KEY, FOREIGNKEY, and CHECK. (The application may vary as per the subject teacher's requirement.)

Application

Student–Department Management System

Department Table

```
CREATE TABLE Department (
    Dept_ID INT PRIMARY KEY,
    Dept_Name VARCHAR(50) UNIQUE NOT NULL
);
```

Student Table

```
CREATE TABLE Student (
    Stud_ID INT PRIMARY KEY,
    Stud_Name VARCHAR(50) NOT NULL,
    Age INT CHECK (Age >= 18),
    Email VARCHAR(100) UNIQUE,
    Dept_ID INT,
    FOREIGN KEY (Dept_ID) REFERENCES Department(Dept_ID)
);
```

INSERT Data into Tables

INSERT INTO **Department** VALUES
(1, 'Computer Engineering'),
(2, 'Information Technology'),
(3, 'Electronics');

```
mysql> SELECT * FROM Department;
+---------+ ----------------------+
| Dept_ID | Dept_Name             |
+---------+ ----------------------+
|       1 | Computer Engineering  |
|       3 | Electronics           |
|       2 | Information Technology |
+------+---------------+
3 rows in set (0.00 sec)
```

INSERT INTO **Student** VALUES

(101, 'Amit Sharma', 20, 'amit@gmail.com', 1),
(102, 'Neha Patil', 19, 'neha@gmail.com', 2),
(103, 'Rahul Verma', 21, 'rahul@gmail.com', 1);

```
mysql> SELECT * FROM Student;
+------+---------+----+------------------+------+
| Stud_ID | Stud_Name   | Age  | Email          | Dept_ID |
+------+---------+----+------------------+------+
|    101 | Amit Sharma |   20 | amit@gmail.com  |       1 |
|    102 | Neha Patil  |   19 | neha@gmail.com  |       2 |
|    103 | Rahul Verma |   21 | rahul@gmail.com |       1 |
+------+---------+----+------------------+------+
```

3 rows in set (0.00 sec)

ALTER TABLE

```
mysql> ALTER TABLE Student ADD City VARCHAR(30);
Query OK, 0 rows affected (0.18 sec)
Records: 0  Duplicates: 0  Warnings: 0

mysql> desc Student;
+--------+----------+----+---+-------+-----+
| Field     | Type         | Null | Key | Default | Extra |
+--------+----------+----+---+-------+-----+
| Stud_ID    | int          | NO    | PRI | NULL     |        |
| Stud_Name | varchar(50)  | NO    |      | NULL     |        |
| Age        | int          | YES  |      | NULL     |        |
| Email      | varchar(100) | YES  | UNI | NULL     |        |
| Dept_ID    | int          | YES  | MUL | NULL     |        |
| City       | varchar(30)  | YES  |      | NULL     |        |
+--------+----------+----+---+-------+-----+
6 rows in set (0.00 sec)
```

RENAME TABLE

```
mysql> RENAME TABLE Student TO Student_Details;
Query OK, 0 rows affected (0.22 sec)

mysql> show tables;
+----------------+
| Tables_in_organization |
+----------------+
| department            |
| student_details        |
+----------------+
2 rows in set (0.00 sec)

mysql> select * from student_details;
+------+---------+----+--------------------+------+----+
| Stud_ID | Stud_Name   | Age  | Email          | Dept_ID | City |
+------+---------+----+--------------------+------+----+
|     101 | Amit Sharma |   20 | amit@gmail.com  |       1 | NULL |
|     102 | Neha Patil  |   19 | neha@gmail.com  |       2 | NULL |
|     103 | Rahul Verma |   21 | rahul@gmail.com |       1 | NULL |
+------+---------+----+--------------------+------+----+
3 rows in set (0.00 sec)
```

TRUNCATE TABLE

```
mysql> TRUNCATE TABLE Student_Details;
Query OK, 0 rows affected (0.66 sec)
```

(All records deleted, structure remains)

```
mysql> select * from student_details;
Empty set (0.00 sec)

mysql> desc Student_details;;
+-------+----------+----+---+-------+--------+
| Field     | Type         | Null | Key | Default | Extra |
+-------+----------+----+---+-------+--------+
| Stud_ID    | int          | NO    | PRI | NULL     |        |
| Stud_Name | varchar(50)  | NO    |      | NULL     |        |
| Age        | int          | YES  |      | NULL     |        |
| Email      | varchar(100) | YES  | UNI | NULL     |        |
| Dept_ID    | int          | YES  | MUL | NULL     |        |
| City       | varchar(30)  | YES  |      | NULL     |        |
+-------+----------+----+---+-------+--------+
6 rows in set (0.00 sec)
```

DROP TABLE

```
DROP TABLE Student_Details;
```

Applying Integrity Constraints in SQL

Application: Student–Course Management System

- **NOT NULL**: This constraint tells that we cannot store a null value in a column. That is, if a column is specified as NOT NULL then we will not be able to store null in this particular column any more.
- **UNIQUE**: This constraint when specified with a column, tells that all the values in the column must be unique. That is, the values in any row of a column must not be repeated.
- **PRIMARY KEY**: A primary key is a field which can uniquely identify each row in a table. And this constraint is used to specify a field in a table as primary key.
- **FOREIGN KEY**: A Foreign key is a field which can uniquely identify each row in a another table. And this constraint is used to specify a field as Foreign key.
- **CHECK**: This constraint helps to validate the values of a column to meet a particular condition. That is, it helps to ensure that the value stored in a column meets a specific condition.
- **DEFAULT**: This constraint specifies a default value for the column when no value is specified by the user.

CREATE TABLE with Integrity Constraints

```
mysql> CREATE TABLE Course (
   ->     Course_ID INT PRIMARY KEY,
   ->     Course_Name VARCHAR(50) NOT NULL UNIQUE,
   ->     Duration INT CHECK (Duration >= 1)
   -> );
Query OK, 0 rows affected (0.37 sec)

mysql> show tables;
+----------------+
| Tables_in_organization |
+----------------+
| course                 |
| department             |
| student_details        |
+----------------+
3 rows in set (0.00 sec)
```

```
mysql> CREATE TABLE Student (
    ->     Student_ID INT PRIMARY KEY,
    ->     Student_Name VARCHAR(50) NOT NULL,
    ->     Email VARCHAR(100) UNIQUE,
    ->     Age INT CHECK (Age >= 18),
    ->     Course_ID INT,
    ->     FOREIGN KEY (Course_ID) REFERENCES Course(Course_ID)
    -> );
Query OK, 0 rows affected (0.56 sec)

mysql> show tables;
+----------------+
| Tables_in_organization |
+----------------+
| course                 |
| department             |
| student                |
| student_details        |
+----------------+
4 rows in set (0.00 sec)
```

Constraint	Applied On	Purpose
PRIMARY KEY	Course_ID, Student_ID	Uniquely identifies records
NOT NULL	Course_Name, Student_Name	Prevents empty values
UNIQUE	Course_Name, Email	Avoids duplicate entries
CHECK	Duration >= 1, Age >= 18	Validates data
FOREIGN KEY	Course_ID in Student	Maintains referential integrity

NOT NULL Violation

```
mysql> INSERT INTO Course VALUES (3, NULL, 5);

ERROR 1048 (23000): Column 'Course_Name' cannot be null
mysql>
```

UNIQUE Violation

```
mysql> INSERT INTO Student VALUES (103, 'Ravi', 'amit@gmail.com', 21, 1);

ERROR 1062 (23000): Duplicate entry 'amit@gmail.com' for key 'student.Email'
```

CHECK Violation

```
mysql> INSERT INTO Student VALUES (104, 'Pooja', 'pooja@gmail.com', 16, 1);

ERROR 3819 (HY000): Check constraint 'student_chk_1' is violated.
mysql>
```

FOREIGN KEY Violation

```
mysql> INSERT INTO Student VALUES (105, 'Kiran', 'kiran@gmail.com', 20, 5);

ERROR 1452 (23000): Cannot add or update a child row: a foreign key constraint fails
(`organization`.`student`, CONSTRAINT `student_ibfk_1` FOREIGN KEY (`Course_ID`)
REFERENCES `course` (`Course_ID`))
mysql>
```

Assignment No 3

Design and run SQL queries to demonstrate the following:

a) Data Manipulation (DML) : Use SQL statements to INSERT, UPDATE, and DELETE records. Apply arithmetic, logical, set operators, pattern matching, and string functions.

b) Access Control (DCL): Use GRANT, REVOKE, and ROLE commands to manage user access.

c) Transaction Control (TCL): Apply START TRANSACTION, COMMIT, ROLLBACK, and SAVEPOINT commands to manage transactions

Application: Library Management System

Objective

To design and run SQL queries demonstrating **Data Manipulation Language (DML)** operations such as **INSERT, UPDATE, DELETE** and apply **arithmetic, logical, set operators, pattern matching, and string functions**.

```
mysql> CREATE TABLE Book (
    ->     Book_ID INT PRIMARY KEY,
    ->     Book_Name VARCHAR(50),
    ->     Author VARCHAR(50),
    ->     Price INT,
    ->     Category VARCHAR(30)
    -> );
Query OK, 0 rows affected (0.29 sec)

mysql> CREATE TABLE Issued_Book (
    ->     Book_Name VARCHAR(50)
    -> );
Query OK, 0 rows affected (0.34 sec)

mysql> INSERT INTO Book VALUES
    -> (1, 'Database Systems', 'Navathe', 500, 'Education'),
    -> (2, 'Operating System', 'Silberschatz', 650, 'Education'),
    -> (3, 'C Programming', 'Dennis Ritchie', 300, 'Programming'),
    -> (4, 'Python Basics', 'Guido', 400, 'Programming');
Query OK, 4 rows affected (0.10 sec)
Records: 4  Duplicates: 0  Warnings: 0

mysql>
```

```
mysql> show tables;
+----------------+
| Tables_in_organization |
+----------------+
| book                   |
| course                 |
| department             |
| issued_book            |
| student                |
| student_details        |
+----------------+
6 rows in set (0.00 sec)

mysql> select * from Book;
+------+------------+-----------+-----+---------+
| Book_ID | Book_Name        | Author        | Price | Category     |
+------+------------+-----------+-----+---------+
|       1 | Database Systems | Navathe        |   500 | Education    |
|       2 | Operating System | Silberschatz    |   650 | Education    |
|       3 | C Programming     | Dennis Ritchie |   300 | Programming |
|       4 | Python Basics     | Guido          |   400 | Programming |
+------+------------+-----------+-----+---------+
4 rows in set (0.00 sec)

mysql>
```

UPDATE Records (DML – UPDATE)
Increase price of Programming books by 10% (Arithmetic Operator)

```
mysql> UPDATE Book
    -> SET Price = Price + (Price * 0.10)
    -> WHERE Category = 'Programming';
Query OK, 2 rows affected (0.07 sec)
Rows matched: 2  Changed: 2  Warnings: 0

mysql> select * from Book;
+------+------------+------------------+-----+---------+
| Book_ID | Book_Name        | Author         | Price | Category    |
+------+------------+------------------+-----+---------+
|       1 | Database Systems | Navathe        |   500 | Education   |
|       2 | Operating System | Silberschatz   |   650 | Education   |
|       3 | C Programming    | Dennis Ritchie |   330 | Programming |
|       4 | Python Basics    | Guido          |   440 | Programming |
+------+------------+------------------+-----+---------+
4 rows in set (0.00 sec)

mysql>
```

DELETE Records (DML – DELETE)
Delete books costing less than 350

```
mysql> DELETE FROM Book
    -> WHERE Price < 350;
Query OK, 1 row affected (0.07 sec)

mysql> select * from Book;
+------+------------+----------------+-----+---------+
| Book_ID | Book_Name        | Author       | Price | Category    |
+------+------------+----------------+-----+---------+
|      1 | Database Systems | Navathe      |   500 | Education   |
|      2 | Operating System | Silberschatz |   650 | Education   |
|      4 | Python Basics    | Guido        |   440 | Programming |
+------+------------+----------------+-----+---------+
3 rows in set (0.00 sec)

mysql>
```

APPLY ARITHMETIC OPERATORS

```
mysql> SELECT Book_Name, Price, Price + 50 AS New_Price
    -> FROM Book;
+-----------+-----+-------+
| Book_Name        | Price | New_Price |
+-----------+-----+-------+
| Database Systems |   500 |       550 |
| Operating System |   650 |       700 |
| Python Basics    |   440 |       490 |
+-----------+-----+-------+
3 rows in set (0.00 sec)
```

APPLY LOGICAL OPERATORS (AND, OR, NOT)

```
mysql> SELECT * FROM Book
    -> WHERE Category = 'Education' AND Price > 500;
+------+------------+----------------+-----+--------+
| Book_ID | Book_Name        | Author       | Price | Category  |
+------+------------+----------------+-----+--------+
|      2 | Operating System | Silberschatz |   650 | Education |
+------+------------+----------------+-----+--------+
1 row in set (0.00 sec)
```

PATTERN MATCHING (LIKE Operator)
Books starting with ‘D’

```
mysql> SELECT * FROM Book
   -> WHERE Book_Name LIKE 'D%';
+------+------------+-----------+-----+--------+
| Book_ID | Book_Name        | Author  | Price | Category  |
+------+------------+-----------+-----+--------+
|      1 | Database Systems | Navathe |   500 | Education |
+------+------------+-----------+-----+--------+
1 row in set (0.00 sec)
```

STRING FUNCTIONS
a) UPPER() and LOWER()

```
mysql> SELECT UPPER(Book_Name) AS Book_Upper,
    ->       LOWER(Author) AS Author_Lower
    -> FROM Book;
+-----------+---------+
| Book_Upper       | Author_Lower |
+-----------+---------+
| DATABASE SYSTEMS | navathe      |
| OPERATING SYSTEM | silberschatz |
| PYTHON BASICS    | guido        |
+-----------+---------+
3 rows in set (0.00 sec)
```

LENGTH()

```
mysql> SELECT Book_Name, LENGTH(Book_Name) AS Name_Length
    -> FROM Book;
+-----------+--------+
| Book_Name        | Name_Length |
+-----------+--------+
| Database Systems |          16 |
| Operating System |          16 |
| Python Basics    |          13 |
+-----------+--------+
3 rows in set (0.00 sec)
```

SUBSTRING()

```
mysql> SELECT SUBSTRING(Book_Name, 1, 4) AS Short_Name
    -> FROM Book;
+--------+
| Short_Name |
+--------+
| Data       |
| Oper       |
| Pyth       |
+--------+
3 rows in set (0.00 sec)
```

SET OPERATORS
Insert Issued Books

```
mysql> INSERT INTO Issued_Book VALUES
    -> ('Database Systems'),
    -> ('Python Basics');
Query OK, 2 rows affected (0.05 sec)
Records: 2  Duplicates: 0  Warnings: 0

mysql> select * from Issued_Book;
+------------+
| Book_Name        |
+------------+
| Database Systems |
| Python Basics    |
+------------+
2 rows in set (0.00 sec)
```

UNION

```
mysql> SELECT Book_Name FROM Book
   -> UNION
   -> SELECT Book_Name FROM Issued_Book;
+ -----------------+
| Book_Name        |
+ -----------------+
| Database Systems |
| Operating System |
| Python Basics    |
+ -----------------+
3 rows in set (0.00 sec)
```

INTERSECT

```
mysql> SELECT Book_Name FROM Book
   -> INTERSECT
   -> SELECT Book_Name FROM Issued_Book;
+ -----------------+
| Book_Name        |
+ -----------------+
| Database Systems |
| Python Basics    |
+ -----------------+
2 rows in set (0.00 sec)
```

EXCEPT / MINUS

```
mysql> SELECT Book_Name FROM Book
   -> EXCEPT
   -> SELECT Book_Name FROM Issued_Book;
+- - - - - - - - - - - -+
| Book_Name        |
+- - - - - - - - - - - -+
| Operating System |
+- - - - - - - - - - - -+
1 row in set (0.00 sec)
```

Access Control (DCL): Use GRANT, REVOKE, and ROLE commands to manage user access.

Objective

To demonstrate **Data Control Language (DCL)** commands using **GRANT**, **REVOKE**, and **ROLE** to manage user access to database objects.

Application: College Student Database Access Control

Database & Table

We will manage access for **Admin**, **Faculty**, and **Student** users.

Step 1: Create Database & Table (Admin)

```
CREATE DATABASE college_db;
USE college_db;

CREATE TABLE student (
    roll_no INT PRIMARY KEY,
    name VARCHAR(50),
    marks INT
);
```

Insert Data

```
INSERT INTO student VALUES
(1,'Amit',85),
(2,'Neha',90),
(3,'Ravi',78);
```

Step 2: Create Users (MySQL)

```
CREATE USER 'admin1'@'localhost' IDENTIFIED BY 'admin123';
CREATE USER 'faculty1'@'localhost' IDENTIFIED BY 'faculty123';
CREATE USER 'student1'@'localhost' IDENTIFIED BY 'student123';
```

Step 3: Create ROLES (MySQL 8.0+)

```
CREATE  ROLE  'admin_role';
CREATE  ROLE  'faculty_role';
CREATE ROLE 'student_role';
```

Step 4: GRANT Privileges to ROLES
Admin Role – Full Access

```
GRANT ALL PRIVILEGES ON college_db.* TO 'admin_role';
```

Faculty Role – Read & Update Marks

```
GRANT SELECT, UPDATE ON college_db.student TO 'faculty_role';
```

Student Role – Read Only

```
GRANT SELECT ON college_db.student TO 'student_role';
```

Step 5: Assign ROLES to Users

```
GRANT 'admin_role' TO 'admin1'@'localhost';
GRANT 'faculty_role' TO 'faculty1'@'localhost';
GRANT 'student_role' TO 'student1'@'localhost';
```

Activate Role (Mandatory)

```
SET DEFAULT ROLE ALL TO
'admin1'@'localhost',
'faculty1'@'localhost',
'student1'@'localhost';
```

Step 6: Check Granted Privileges (Output)

```
SHOW GRANTS FOR 'faculty1'@'localhost';
```

```
mysql> SHOW GRANTS FOR 'faculty1'@'localhost';
+-----------------------------------+
| Grants for faculty1@localhost                    |
+-----------------------------------+
| GRANT USAGE ON *.* TO `faculty1`@`localhost`       |
| GRANT `faculty_role`@`%` TO `faculty1`@`localhost` |
+-----------------------------------+
2 rows in set (0.00 sec)
```

Step 7: Login / Switch Between Users (Windows)

Exit Current User

Login as Admin

Terminal 1 (Bash cmd)

C:\>cd C:\Program Files\MySQL\MySQL Server 8.0\bin
C:\Program Files\MySQL\MySQL Server 8.0\bin>mysql -u **root** -p
Enter password: ****

Login as Faculty

Terminal 2 (Bash cmd)

C:\Program Files\MySQL\MySQL Server 8.0\bin>mysql -u **faculty1** -p
Enter password: **********
Password (faculty123)

Login as Student

Terminal 3(Bash cmd)

C:\Program Files\MySQL\MySQL Server 8.0\bin>mysql -u **student1** -p
Enter password: **********
Password (student123)
Privileges: Select

Step 8: Test Access (Input & Output)
Student User

```
mysql> SELECT * FROM college_db.student;
+------+-----+-----+
| roll_no | name  | marks |
+------+-----+-----+
|      1 | Amit  |    85 |
|      2 | Neha  |    90 |
|      3 | Ravi  |    78 |
|      4 | Pooja |    88 |
+------+-----+-----+
4 rows in set (0.00 sec)
```

```
mysql> INSERT INTO college_db.student VALUES (4,'Pooja',88);
ERROR 1142 (42000): INSERT command denied to user 'student1'@'localhost' for
table 'student'
```

✕ **ERROR: Access denied**

Step 9: REVOKE Privileges

Remove UPDATE from Faculty

```
REVOKE UPDATE ON college_db.student FROM 'faculty_role';
```

Check Again

```
SHOW GRANTS FOR 'faculty1'@'localhost';
```

Step 10: Drop Role / User (Optional)

```
DROP ROLE 'student_role';
DROP USER 'student1'@'localhost';
```

Summary

Command	Purpose
GRANT	Give permissions
REVOKE	Remove permissions
ROLE	Group permissions
SHOW GRANTS	Check access
mysql -u user -p	Switch user

Transaction Control (TCL): Apply START TRANSACTION, COMMIT, ROLLBACK, and SAVEPOINT commands to manage transactions

Application: Bank Account Transaction System

```
mysql> CREATE TABLE Account (
   ->     Acc_No INT PRIMARY KEY,
   ->     Acc_Holder VARCHAR(50),
   ->     Balance INT
   -> );
Query OK, 0 rows affected (0.37 sec)
```

```
mysql> INSERT INTO Account VALUES
   -> (101, 'Amit Patil', 10000),
   -> (102, 'Neha Sharma', 15000);
Query OK, 2 rows affected (0.05 sec)
Records: 2  Duplicates: 0  Warnings: 0

mysql> select * from Account;
+-------+-----------+---------+
| Acc_No | Acc_Holder  | Balance |
+-------+-----------+---------+
|    101 | Amit Patil  |   10000 |
|    102 | Neha Sharma |   15000 |
+-------+-----------+---------+
2 rows in set (0.00 sec)
```

```
mysql> START TRANSACTION;
Query OK, 0 rows affected (0.00 sec)

mysql> UPDATE Account
    -> SET Balance = Balance - 2000
    -> WHERE Acc_No = 101;
Query OK, 1 row affected (0.02 sec)
Rows matched: 1  Changed: 1  Warnings: 0

mysql> UPDATE Account
    -> SET Balance = Balance + 2000
    -> WHERE Acc_No = 102;
Query OK, 1 row affected (0.00 sec)
Rows matched: 1  Changed: 1  Warnings: 0

mysql> SAVEPOINT transfer_done;
Query OK, 0 rows affected (0.00 sec)
```

```
mysql> select * from Account;
+------+----------+---------+
| Acc_No | Acc_Holder  | Balance |
+------+----------+---------+
|    101 | Amit Patil  |    8000 |
|    102 | Neha Sharma |   17000 |
+------+----------+---------+
```

2 row in set (0.00 sec)

```
mysql> UPDATE Account
    -> SET Balance = Balance - 5000
    -> WHERE Acc_No = 101;
Query OK, 1 row affected (0.00 sec)
Rows matched: 1  Changed: 1  Warnings: 0

mysql> ROLLBACK TO transfer_done;
Query OK, 0 rows affected (0.02 sec)

mysql> select * from Account;
+------+----------+---------+
| Acc_No | Acc_Holder  | Balance |
+------+----------+---------+
|    101 | Amit Patil  |    8000 |
|    102 | Neha Sharma |   17000 |
+------+----------+---------+
2 rows in set (0.00 sec)
```

```
mysql> COMMIT;
Query OK, 0 rows affected (0.04 sec)

mysql> select * from Account;
+------+----------+---------+
| Acc_No | Acc_Holder  | Balance |
+------+----------+---------+
|    101 | Amit Patil  |    8000 |
|    102 | Neha Sharma |   17000 |
+------+----------+---------+
2 rows in set (0.00 sec)
```

Savitribai Phule Pune University
Second Year of Computer Engineering & Computer Science and Engineering
(2024 Course)

PCC-254-COM : Database Management Laboratory

Assignment 4

Aggregate Functions and Grouping

Use aggregate functions along with GROUPBY and HAVING clauses to retrieve summarized data from the database.

Application: Student Result Analysis System

```
mysql> CREATE TABLE Student_Result (
    ->     Student_ID INT,
    ->     Student_Name VARCHAR(50),
    ->     Subject VARCHAR(30),
    ->     Marks INT,
    ->     Department VARCHAR(30)
    -> );
Query OK, 0 rows affected (0.93 sec)
```

```
INSERT INTO Student_Result VALUES
(1, 'Amit', 'DBMS', 78, 'Computer'),
(2, 'Neha', 'DBMS', 85, 'Computer'),
(3, 'Rahul', 'OS', 72, 'Computer'),
(4, 'Sneha', 'OS', 88, 'IT'),
(5, 'Pooja', 'DBMS', 91, 'IT'),
(6, 'Ravi', 'OS', 65, 'IT');
```

Aggregate Functions

a) COUNT – Number of Students in Each Department

```
mysql> SELECT Department, COUNT(Student_ID) AS Total_Students
    -> FROM Student_Result
    -> GROUP BY Department;
+--------+-----------+
| Department | Total_Students |
+--------+-----------+
| Computer   |           3 |
| IT         |           3 |
+--------+-----------+
2 rows in set (0.00 sec)
```

b) AVG – Average Marks per Subject

```
mysql> SELECT Subject, AVG(Marks) AS Avg_Marks
   -> FROM Student_Result
   -> GROUP BY Subject;
+------+-------+
| Subject | Avg_Marks |
+------+-------+
| DBMS    |   84.6667 |
| OS      |   75.0000 |
+------+-------+
2 rows in set (0.02 sec)
```

c) **MAX and MIN Marks per Department**

```
mysql> SELECT Department,
   ->       MAX(Marks) AS Highest_Marks,
   ->       MIN(Marks) AS Lowest_Marks
   -> FROM Student_Result
   -> GROUP BY Department;
+--------+----------+---------+
| Department | Highest_Marks | Lowest_Marks |
+--------+----------+---------+
| Computer   |          85 |          72 |
| IT         |          91 |          65 |
+--------+----------+---------+
2 rows in set (0.02 sec)
```

d) SUM of Marks per Department

```
mysql> SELECT Department, SUM(Marks) AS Total_Marks
    -> FROM Student_Result
    -> GROUP BY Department;
+--------+---------+
| Department | Total_Marks |
+--------+---------+
| Computer   |       235 |
| IT         |       244 |
+--------+---------+
2 rows in set (0.00 sec)
```

GROUP BY with HAVING Clause

Departments with Average Marks Greater Than 80

```
mysql> SELECT Department, AVG(Marks) AS Avg_Marks
    -> FROM Student_Result
    -> GROUP BY Department
    -> HAVING AVG(Marks) > 80;
+-------+-------+
| Department | Avg_Marks |
+-------+-------+
| IT         |   81.3333 |
+-------+-------+
1 row in set (0.02 sec)
```

Subjects Having More Than 2 Students

```
mysql> SELECT Subject, COUNT(Student_ID) AS Student_Count
    -> FROM Student_Result
    -> GROUP BY Subject
    -> HAVING COUNT(Student_ID) > 2;
+------+----------+
| Subject | Student_Count |
+------+----------+
| DBMS    |            3 |
| OS      |            3 |
+------+----------+
2 rows in set (0.00 sec)
```

GROUP BY with WHERE and HAVING

```
mysql> SELECT Department, AVG(Marks) AS Avg_Marks
    -> FROM Student_Result
    -> WHERE Marks >= 70
    -> GROUP BY Department
    -> HAVING AVG(Marks) > 75;
+--------+--------+
| Department | Avg_Marks |
+--------+--------+
| Computer   |   78.3333 |
| IT         |   89.5000 |
+--------+--------+
2 rows in set (0.00 sec)
```

Savitribai Phule Pune University
Second Year of Computer Engineering & Computer Science and Engineering
(2024 Course)

PCC-254-COM : Database Management Laboratory

Assignment 5

JOIN Operations and Views

Perform various types of JOIN operations to extract meaningful relationships between tables. Create and manage different database views.

Application: Online Shopping (E-Commerce) Database

Database Design
CUSTOMER Table

```
mysql> CREATE TABLE Customer (
   ->     customer_id INT PRIMARY KEY,
   ->     customer_name VARCHAR(50),
   ->     city VARCHAR(30)
   -> );
Query OK, 0 rows affected (0.65 sec)
```

PRODUCT Table

```
mysql> CREATE TABLE Product (
   ->     product_id INT PRIMARY KEY,
   ->     product_name VARCHAR(50),
   ->     price DECIMAL(10,2)
   -> );
Query OK, 0 rows affected (0.34 sec)
```

ORDER Table

```
mysql> CREATE TABLE Orders (
   ->     order_id INT PRIMARY KEY,
   ->     customer_id INT,
   ->     product_id INT,
   ->     quantity INT,
   ->     FOREIGN KEY (customer_id) REFERENCES Customer(customer_id),
   ->     FOREIGN KEY (product_id) REFERENCES Product(product_id)
   -> );
Query OK, 0 rows affected (1.29 sec)
```

```
mysql> INSERT INTO Customer VALUES
    -> (1, 'Amit', 'Pune'),
    -> (2, 'Neha', 'Mumbai'),
    -> (3, 'Rahul', 'Nagpur');
Query OK, 3 rows affected (0.24 sec)
Records: 3  Duplicates: 0  Warnings: 0

mysql> INSERT INTO Product VALUES
    -> (101, 'Laptop', 55000),
    -> (102, 'Mobile', 20000),
    -> (103, 'Headphones', 2500);
Query OK, 3 rows affected (0.06 sec)
Records: 3  Duplicates: 0  Warnings: 0

mysql>
mysql> INSERT INTO Orders VALUES
    -> (1001, 1, 101, 1),
    -> (1002, 2, 102, 2),
    -> (1003, 1, 103, 3);
Query OK, 3 rows affected (0.07 sec)
Records: 3  Duplicates: 0  Warnings: 0
```

JOIN Operations (Application-Based Queries)

a) INNER JOIN

Display customer name, product name, and quantity ordered

```
mysql> SELECT customer_name, product_name, quantity
    -> FROM Orders
    -> INNER JOIN Customer ON Orders.customer_id = Customer.customer_id
    -> INNER JOIN Product ON Orders.product_id = Product.product_id;
+----------+----------+-------+
| customer_name | product_name | quantity |
+----------+----------+-------+
| Amit          | Laptop       |        1 |
| Neha          | Mobile       |        2 |
| Amit          | Headphones   |        3 |
+----------+----------+-------+
3 rows in set (0.01 sec)
```

b) LEFT JOIN

List all customers and their orders (including customers with no orders)

```
mysql> SELECT customer_name, order_id, product_name
    -> FROM Customer
    -> LEFT JOIN Orders ON Customer.customer_id = Orders.customer_id
    -> LEFT JOIN Product ON Orders.product_id = Product.product_id;
+-----------+--------+-----------+
| customer_name | order_id | product_name |
+-----------+--------+-----------+
| Amit          |     1001 | Laptop       |
| Amit          |     1003 | Headphones   |
| Neha          |     1002 | Mobile       |
| Rahul         |     NULL | NULL         |
+-----------+--------+-----------+
4 rows in set (0.01 sec)
```

c) RIGHT JOIN
Show all products including products not yet ordered

```
mysql> SELECT product_name, customer_name
    -> FROM Orders
    -> RIGHT JOIN Product ON Orders.product_id = Product.product_id
    -> LEFT JOIN Customer ON Orders.customer_id = Customer.customer_id;
+-----------+-----------+
| product_name | customer_name |
+-----------+-----------+
| Laptop       | Amit          |
| Mobile       | Neha          |
| Headphones   | Amit          |
+-----------+-----------+
3 rows in set (0.00 sec)
```

d) FULL OUTER JOIN

(Oracle / PostgreSQL)

```
mysql> SELECT customer_name, product_name
   -> FROM Customer
   -> FULL OUTER JOIN Orders
   -> ON Customer.customer_id = Orders.customer_id
   -> FULL OUTER JOIN Product
   -> ON Orders.product_id = Product.product_id;
ERROR 1064 (42000): You have an error in your SQL syntax; check the manual that
corresponds to your MySQL server version for the right syntax to use near 'OUTER
JOIN Orders
ON Customer.customer_id = Orders.customer_id
FULL OUTER JOIN P' at line 3
```

e) CROSS JOIN

Generate all possible customer–product combinations

```
mysql> SELECT customer_name, product_name
    -> FROM Customer
    -> CROSS JOIN Product;
+----------+----------+
| customer_name | product_name |
+----------+----------+
| Rahul        | Laptop       |
| Neha         | Laptop       |
| Amit         | Laptop       |
| Rahul        | Mobile       |
| Neha         | Mobile       |
| Amit         | Mobile       |
| Rahul        | Headphones   |
| Neha         | Headphones   |
| Amit         | Headphones   |
+----------+----------+
9 rows in set (0.00 sec)
```

f) SELF JOIN
Customers from the same city

```
mysql> SELECT A.customer_name AS
Customer1, B.customer_name AS Customer2,
A.city
    -> FROM Customer A
    -> JOIN Customer B
    -> ON A.city = B.city
    -> AND A.customer_id <> B.customer_id;
Empty set (0.04 sec)
```

4. Database Views

a) Create View – Order Details

```
mysql> CREATE VIEW Order_Details_View AS
    -> SELECT order_id, customer_name, product_name, quantity, price,
    -> (quantity * price) AS total_amount
    -> FROM Orders
    -> JOIN Customer ON Orders.customer_id = Customer.customer_id
    -> JOIN Product ON Orders.product_id = Product.product_id;
Query OK, 0 rows affected (0.12 sec)

mysql> show tables;
+-------------+
| Tables_in_db1      |
+-------------+
| customer           |
| order_details_view |
| orders             |
| product            |
+-------------+
4 rows in set (0.06 sec)
```

b) Display View

```
mysql> SELECT * FROM Order_Details_View;
+-------+----------+---------+-------+-------+---------+
| order_id | customer_name | product_name | quantity | price     | total_amount |
+-------+----------+---------+-------+-------+---------+
|     1001 | Amit          | Laptop       |        1 | 55000.00 |     55000.00 |
|     1002 | Neha          | Mobile       |        2 | 20000.00 |     40000.00 |
|     1003 | Amit          | Headphones   |        3 |  2500.00 |      7500.00 |
+-------+----------+---------+-------+-------+---------+
3 rows in set (0.00 sec)

mysql> desc Order_Details_View;
+----------+-----------------+----+---+-------+--------+
| Field        | Type          | Null | Key | Default | Extra |
+----------+-----------------+----+---+-------+--------+
| order_id      | int           | NO   |     | NULL    |       |
| customer_name | varchar(50)   | YES  |     | NULL    |       |
| product_name  | varchar(50)   | YES  |     | NULL    |       |
| quantity      | int           | YES  |     | NULL    |       |
| price         | decimal(10,2) | YES  |     | NULL    |       |
| total_amount  | decimal(20,2) | YES  |     | NULL    |       |
+----------+-----------------+----+---+-------+--------+
6 rows in set (0.03 sec)
```

c) View with Condition
Customers from Pune

```
mysql> CREATE VIEW Pune_Customers AS
    -> SELECT customer_name
    -> FROM Customer
    -> WHERE city = 'Pune';
Query OK, 0 rows affected (0.11 sec)

mysql> show tables;
+-------------+
| Tables_in_db1      |
+-------------+
| customer           |
| order_details_view |
| orders             |
| product            |
| pune_customers     |
+-------------+
5 rows in set (0.00 sec)
```

d) Update Through View

```
mysql> UPDATE Pune_Customers
    -> SET customer_name = 'Amit Kumar'
    -> WHERE customer_name = 'Amit';
Query OK, 1 row affected (0.07 sec)
Rows matched: 1  Changed: 1  Warnings: 0

mysql> select * from Pune_Customers;
+----------+
| customer_name |
+----------+
| Amit Kumar    |
+----------+
1 row in set (0.00 sec)
```

e) Drop View

```
mysql> DROP VIEW Order_Details_View;
Query OK, 0 rows affected (0.10 sec)

mysql> show tables;
+----------+
| Tables_in_db1  |
+----------+
| customer       |
| orders         |
| product        |
| pune_customers |
+----------+
4 rows in set (0.00 sec)
```

Savitribai Phule Pune University
Second Year of Computer Engineering & Computer Science and Engineering
(2024 Course)

PCC-254-COM : Database Management Laboratory

Assignment no 6

- A Subquery or Inner query or a Nested query is a query within another SQL query and embedded within the WHERE clause.

- A subquery is used to return data that will be used in the main query as a condition to further restrict the data to be retrieved.

- Subqueries can be used with the SELECT, INSERT, UPDATE, and DELETE statements along with the operators like =, <, >, >=, <=, IN, BETWEEN, etc.

There are a few rules that subqueries must follow –

- Subqueries must be enclosed within parentheses.
- A subquery can have only one column in the SELECT clause, unless multiple columns are in the main query for the subquery to compare its selected columns.
- An ORDER BY command cannot be used in a subquery, although the main query can use an ORDER BY. The GROUP BY command can be used to perform the same function as the ORDER BY in a subquery.
- Subqueries that return more than one row can only be used with multiple value operators such as the IN operator.

There are a few rules that subqueries must follow –

- The SELECT list cannot include any references to values that evaluate to a BLOB, ARRAY, CLOB, or NCLOB.
- A subquery cannot be immediately enclosed in a set function.
- The BETWEEN operator cannot be used with a subquery. However, the BETWEEN operator can be used within the subquery.

Subqueries with the SELECT Statement

```
SELECT column_name [, column_name ]

FROM table1 [, table2 ]

WHERE column_name OPERATOR

        (SELECT column_name [, column_name ]
        FROM table1 [, table2 ]
        [WHERE])
```

Subqueries

Write and execute subqueries to retrieve data from one table based on results from another.

Application: Online Shopping (E-Commerce) Database

Database Design
CUSTOMER Table

```
mysql> CREATE TABLE Customer (
    ->     customer_id INT PRIMARY KEY,
    ->     customer_name VARCHAR(50),
    ->     city VARCHAR(30)
    -> );
Query OK, 0 rows affected (0.65 sec)
```

PRODUCT Table

```
mysql> CREATE TABLE Product (
    ->     product_id INT PRIMARY KEY,
    ->     product_name VARCHAR(50),
    ->     price DECIMAL(10,2)
    -> );
Query OK, 0 rows affected (0.34 sec)
```

ORDER Table

```
mysql> CREATE TABLE Orders (
    ->     order_id INT PRIMARY KEY,
    ->     customer_id INT,
    ->     product_id INT,
    ->     quantity INT,
    ->     FOREIGN KEY (customer_id) REFERENCES Customer(customer_id),
    ->     FOREIGN KEY (product_id) REFERENCES Product(product_id)
    -> );
Query OK, 0 rows affected (1.29 sec)
```

```
mysql> INSERT INTO Customer VALUES
    -> (1, 'Amit', 'Pune'),
    -> (2, 'Neha', 'Mumbai'),
    -> (3, 'Rahul', 'Nagpur');
Query OK, 3 rows affected (0.24 sec)
Records: 3  Duplicates: 0  Warnings: 0

mysql> INSERT INTO Product VALUES
    -> (101, 'Laptop', 55000),
    -> (102, 'Mobile', 20000),
    -> (103, 'Headphones', 2500);
Query OK, 3 rows affected (0.06 sec)
Records: 3  Duplicates: 0  Warnings: 0

mysql>
mysql> INSERT INTO Orders VALUES
    -> (1001, 1, 101, 1),
    -> (1002, 2, 102, 2),
    -> (1003, 1, 103, 3);
Query OK, 3 rows affected (0.07 sec)
Records: 3  Duplicates: 0  Warnings: 0
```

a) Subquery using IN

Find customers who have placed orders

```
mysql> SELECT customer_name
    -> FROM Customer
    -> WHERE customer_id IN (
    ->     SELECT customer_id
    ->     FROM Orders
    -> );
+----------+
| customer_name |
+----------+
| Amit Kumar    |
| Neha          |
+----------+
2 rows in set (0.00 sec)
```

b) Subquery using NOT IN

Find customers who have NOT placed any order

```
mysql> SELECT customer_name
    -> FROM Customer
    -> WHERE customer_id NOT IN (
    ->     SELECT customer_id
    ->     FROM Orders
    -> );
+----------+
| customer_name |
+----------+
| Rahul         |
+----------+
1 row in set (0.01 sec)
```

c) Subquery using = (Single-Row Subquery)

Find the product with the highest price

```
mysql> SELECT product_name, price
    -> FROM Product
    -> WHERE price = (
    ->     SELECT MAX(price)
    ->     FROM Product
    -> );
+----------+-------+
| product_name | price     |
+----------+-------+
| Laptop       | 55000.00 |
+----------+-------+
1 row in set (0.04 sec)
```

d) Subquery in SELECT Clause

Display order ID and total amount for each order

```
mysql> SELECT order_id,
    ->       (SELECT price FROM Product WHERE Product.product_id = Orders.product_id) * quantity
    ->       AS total_amount
    -> FROM Orders;
+-------+----------+
| order_id | total_amount |
+-------+----------+
|     1001 |     55000.00 |
|     1002 |     40000.00 |
|     1003 |      7500.00 |
+-------+----------+
3 rows in set (0.00 sec)
```

e) Subquery in FROM Clause

Find customers whose total purchase amount is greater than 30,000

```
mysql> SELECT customer_id, total
    -> FROM (
    ->     SELECT Orders.customer_id,
    ->            SUM(Product.price * Orders.quantity) AS total
    ->     FROM Orders
    ->     JOIN Product ON Orders.product_id = Product.product_id
    ->     GROUP BY Orders.customer_id
    -> ) AS Purchase
    -> WHERE total > 30000;
+-----------+-------+
| customer_id | total    |
+-----------+-------+
|           1 | 62500.00 |
|           2 | 40000.00 |
+-----------+-------+
2 rows in set (0.02 sec)
```

f) Correlated Subquery

Find products that are more expensive than the average price

```
mysql> SELECT product_name, price
    -> FROM Product P
    -> WHERE price > (
    ->     SELECT AVG(price)
    ->     FROM Product
    ->     WHERE Product.product_id = P.product_id OR 1=1
    -> );
+---------+-------+
| product_name | price     |
+---------+-------+
| Laptop       | 55000.00 |
+---------+-------+
1 row in set (0.00 sec)
```

g) Subquery with EXISTS

List customers who have ordered at least one product

```
mysql> SELECT customer_name
    -> FROM Customer C
    -> WHERE EXISTS (
    ->     SELECT 1
    ->     FROM Orders O
    ->     WHERE O.customer_id = C.customer_id
    -> );
------+---          +
| customer_name |
------+---          +
| Amit Kumar    |
| Neha          |
------+---          +
2 rows in set (0.00 sec)
```

Assignment 7

Title

Stored Procedure Using Explicit Cursor in PL/SQL

Theory (Brief)

A cursor in PL/SQL is used to process query results **one row at a time**. An explicit cursor is defined and controlled by the programmer. Stored procedures are named PL/SQL blocks stored in the database and used to perform repeated tasks efficiently. Using cursors inside stored procedures allows handling multiple rows with procedural logic.

Problem Statement

Create and execute a **PL/SQL stored procedure using an explicit cursor** to retrieve and display employee details from the Emp1 table.

Procedure / Steps

1. Create the Emp1 table.
2. Insert sample records into the table.
3. Write a stored procedure using an explicit cursor to select employee details.
4. Open the cursor and fetch records one by one.
5. Display the employee ID, name, and salary using DBMS_OUTPUT.

6. Close the cursor.
7. Execute the stored procedure and verify the output.

Program Code

Table Creation

```
CREATE TABLE Emp1 (
   emp_id NUMBER,
   emp_name VARCHAR2(50),
   salary NUMBER
);
```

Insert Records

```
INSERT INTO Emp1 VALUES (1, 'Ram', 50000);
INSERT INTO Emp1 VALUES (2, 'Sham', 60000);
INSERT INTO Emp1 VALUES (3, 'Tom', 55000);
COMMIT;
```

1. Stored Procedure Using Explicit Cursor

```
CREATE OR REPLACE PROCEDURE show_employee_details
IS
   CURSOR emp_cursor IS
      SELECT emp_id, emp_name, salary FROM Emp1;

   v_id Emp1.emp_id%TYPE;
   v_name Emp1.emp_name%TYPE;
   v_salary Emp1.salary%TYPE;
BEGIN
   OPEN emp_cursor;
```

```
    LOOP
        FETCH emp_cursor INTO v_id, v_name, v_salary;
        EXIT WHEN emp_cursor%NOTFOUND;

        DBMS_OUTPUT.PUT_LINE(
            'ID: ' || v_id ||
            ', Name: ' || v_name ||
            ', Salary: ' || v_salary
        );
    END LOOP;

    CLOSE emp_cursor;
END;
/
```

Execution

How to check output

1. Option 1

   ```
   BEGIN
       show_employee_details;
   END;
   /
   ```

2. Option 2

   ```
   SET SERVEROUTPUT ON
   EXEC show_employee_details;
   ```

Output

Employee details are displayed one by one showing employee ID, name, and salary.

Result

The stored procedure using an explicit cursor was successfully created and executed. Employee records were fetched and displayed row by row.

Conclusion

This experiment demonstrates how **explicit cursors** are used within **stored procedures** to process multiple rows of data efficiently in PL/SQL.

****

2. Stored Procedure Using a Parameterized Cursor

```
CREATE OR REPLACE PROCEDURE show_high_salary1(p_salary NUMBER)
IS
   CURSOR emp_cursor IS
      SELECT emp_name, salary
      FROM Emp1
      WHERE salary > p_salary;
   v_name   Emp1.emp_name%TYPE;
   v_salary Emp1.salary%TYPE;
BEGIN
   OPEN emp_cursor;
   LOOP
      FETCH emp_cursor INTO v_name, v_salary;
      EXIT WHEN emp_cursor%NOTFOUND;
      DBMS_OUTPUT.PUT_LINE( v_name || ' earns ' || v_salary  );
   END LOOP;
   CLOSE emp_cursor;
END;
/
```

How to check output

1. Option 1

   EXEC show_high_salary1(50000);

2. Option 2

   BEGIN

   show_high_salary1(50000); END;

   /

---

3. Function Using Cursor (Returns Total Salary)

```
CREATE OR REPLACE FUNCTION total_salary1 RETURN

NUMBER

IS

   CURSOR emp_cursor IS SELECT

      salary FROM Emp1;


   v_salary Emp1.salary%TYPE;

   v_total  NUMBER := 0;

BEGIN

   OPEN emp_cursor;


   LOOP

      FETCH emp_cursor INTO v_salary; EXIT

      WHEN emp_cursor%NOTFOUND;


      v_total := v_total + v_salary; END

   LOOP;
```

```
   CLOSE emp_cursor;


   RETURN v_total;
END;
/
```

How to check output……

1. Option1

```
BEGIN
   DBMS_OUTPUT.PUT_LINE('Total Salary: ' || total_salary1); END;
/
```

2. Option 2

```
DECLARE
   v_total NUMBER;
BEGIN
   v_total := total_salary1;
   DBMS_OUTPUT.PUT_LINE(v_total); END;
/
```

3. Option 3

```
SELECT total_salary1 FROM dual;
```

Assignment 8

Database Triggers

Title : Implementation of Database Triggers to Maintain Data Integrity

Aim:

To implement and test a **database trigger** that maintains data integrity by validating data before insertion or update in a database table.

Theory (Brief)

A database trigger is a stored PL/SQL block that is automatically executed when a specified event occurs on a table, such as INSERT, UPDATE, or DELETE. Triggers help enforce business rules, maintain data integrity, and provide automatic validation at the database level.

Problem Statement

Design and implement a **database trigger** to validate employee salary values and prevent insertion or updating of invalid salary data in the employees table.

Procedure / Steps

1. Create the employees table.
2. Write a trigger that fires **before INSERT or UPDATE** on the table.
3. In the trigger, check whether the salary value is valid.
4. Raise an error if the salary is less than or equal to zero.
5. Test the trigger using valid and invalid data.
6. Observe and verify the output.

Program Code

Table Creation

```
CREATE TABLE employees2 (
   emp_id NUMBER PRIMARY KEY,
   emp_name VARCHAR2(50),
   salary NUMBER
);
```

1. Trigger to Maintain Data Integrity

```
CREATE OR REPLACE TRIGGER trg_check_salary
BEFORE INSERT OR UPDATE ON employees2
FOR EACH ROW
BEGIN
   IF :NEW.salary <= 0 THEN
      RAISE_APPLICATION_ERROR(-20001, 'Salary must be greater than zeroAAAA' );
   END IF;
END;
/
```

Testing the Trigger

Valid Operation

INSERT INTO employees VALUES (1, 'Ram', 50000);

Invalid Operation

INSERT INTO employees VALUES (2, 'Sham', -30000);

Expected Output

- Valid data is inserted successfully.
- Invalid salary values are rejected with an error message.

Result

The database trigger was successfully created and tested. It prevented insertion or update of invalid salary values, thereby maintaining data integrity.

Conclusion

This experiment demonstrates the use of **database triggers** to automatically enforce data validation rules and ensure consistency and integrity of data in the database.

Viva Voce Questions (Sample)

1. What is a database trigger?
2. Difference between BEFORE and AFTER triggers?
3. What is :NEW and :OLD in triggers?
4. Why are triggers used for data integrity?

2. Trigger to Prevent Deleting Employees with High Salary

Business Rule

Employees earning more than 50,000 cannot be deleted.

Trigger Code

```
CREATE OR REPLACE TRIGGER trg_no_delete_high_salary
BEFORE DELETE ON employees
FOR EACH ROW
BEGIN
   IF :OLD.salary > 50000 THEN
```

```
      RAISE_APPLICATION_ERROR(-20002,
      'Cannot delete employees with salary greater than 50000');
   END IF;
END;
/
```

Test

```
DELETE FROM employees WHERE emp_id = 101;
```

3. Trigger to Automatically Log Employee Inserts (Audit Integrity)

Audit Table

```
CREATE TABLE emp_audit (
   emp_id NUMBER,
   action VARCHAR2(20),
   action_date DATE
);
```

Trigger Code

```
CREATE OR REPLACE TRIGGER trg_emp_audit
AFTER INSERT ON employees
FOR EACH ROW
BEGIN
   INSERT INTO emp_audit
   VALUES (:NEW.emp_id, 'INSERT', SYSDATE);
END;
/
```

Test

```
INSERT INTO employees VALUES (102, 'Mary', 40000);
SELECT * FROM emp_audit;
```

1. Trigger to Maintain Referential Integrity (Manual FK Control)

Business Rule

A department cannot be deleted if employees exist in it.

Trigger Code

```
CREATE OR REPLACE TRIGGER
trg_dept_delete BEFORE DELETE ON
departments
FOR EACH
ROW
DECLARE
   v_count
NUMBER; BEGIN
   SELECT COUNT(*) INTO
   v_count FROM employees
   WHERE dept_id = :OLD.dept_id;

   IF v_count > 0 THEN
      RAISE_APPLICATION_ERROR(-20003,
      'Department has employees. Cannot delete.');
   END IF;
END;
/
```

5. Trigger to Automatically Set Default Salary

Business Rule

If salary is not provided, set it to 30000. Trigger Code

```
CREATE OR REPLACE TRIGGER trg_default_salary
BEFORE INSERT ON employees
FOR EACH
ROW BEGIN
   IF :NEW.salary IS NULL THEN
      :NEW.salary := 30000;
   END IF;
END;
/
```

MongoDB Assignment

Assignment No 09

Statement: CRUD Operations using MongoDB Design and implement basic Create, Read, Update, and Delete (CRUD) operations using MongoDB. Use the save method and logical operators where necessary

1. Create / Switch to Database

```
use Empdb1
```

2. Create Employee Collection

```
db.createCollection("Employee")
```

3. Insert 10 Documents into Employee Collection

```
db.Employee.insertMany([
  {
    Empid: 101,
    Name: { FName: "Amit", LName: "Sharma" },
    CompanyName: "TCS",
    Salary: 65000,
    Designation: "Software Engineer",
    Age: 25,
    Expertise: ["Java", "Spring", "SQL"],
    DOB: ISODate("1999-02-15"),
    EmailId: "amit.sharma@tcs.com",
    Contact: "9876543210"
```

```
},
{
  Empid: 102,
  Name: { FName: "Neha", LName: "Patil" },
  CompanyName: "Infosys",
  Salary: 72000,
  Designation: "System Analyst",
  Age: 27,
  Expertise: ["Python", "Django", "MongoDB"],
  DOB: ISODate("1998-06-10"),
  EmailId: "neha.patil@infosys.com",
  Contact: "9876543211"
},
{
  Empid: 103,
  Name: { FName: "Rahul", LName: "Verma" },
  CompanyName: "Wipro",
  Salary: 58000,
  Designation: "Developer",
  Age: 24,
  Expertise: ["C++", "DSA"],
  DOB: ISODate("2000-01-05"),
  EmailId: "rahul.verma@wipro.com",
  Contact: "9876543212"
},
{
  Empid: 104,
  Name: { FName: "Pooja", LName: "Kulkarni" },
  CompanyName: "Capgemini",
  Salary: 69000,
  Designation: "Backend Developer",
  Age: 26,
  Expertise: ["NodeJS", "Express", "MongoDB"],
  DOB: ISODate("1999-08-20"),
  EmailId: "pooja.k@capgemini.com",
  Contact: "9876543213"
},
{
  Empid: 105,
  Name: { FName: "Suresh", LName: "Rao" },
  CompanyName: "IBM",
  Salary: 85000,
  Designation: "Senior Engineer",
  Age: 30,
  Expertise: ["Cloud", "AWS", "DevOps"],
  DOB: ISODate("1995-11-12"),
  EmailId: "suresh.rao@ibm.com",
  Contact: "9876543214"
},
{
  Empid: 106,
  Name: { FName: "Anjali", LName: "Mehta" },
  CompanyName: "Accenture",
```

```
    Salary: 76000,
    Designation: "Consultant",
    Age: 28,
    Expertise: ["SAP", "ERP"],
    DOB: ISODate("1997-03-25"),
    EmailId: "anjali.mehta@accenture.com",
    Contact: "9876543215"
  },
  {
    Empid: 107,
    Name: { FName: "Kiran", LName: "Joshi" },
    CompanyName: "HCL",
    Salary: 62000,
    Designation: "Support Engineer",
    Age: 26,
    Expertise: ["Linux", "Networking"],
    DOB: ISODate("1998-12-18"),
    EmailId: "kiran.joshi@hcl.com",
    Contact: "9876543216"
  },
  {
    Empid: 108,
    Name: { FName: "Rohit", LName: "Deshmukh" },
    CompanyName: "Tech Mahindra",
    Salary: 70000,
    Designation: "Full Stack Developer",
    Age: 27,
    Expertise: ["React", "NodeJS", "MySQL"],
    DOB: ISODate("1998-09-09"),
    EmailId: "rohit.d@techmahindra.com",
    Contact: "9876543217"
  },
  {
    Empid: 109,
    Name: { FName: "Sneha", LName: "Iyer" },
    CompanyName: "Oracle",
    Salary: 90000,
    Designation: "Database Admin",
    Age: 31,
    Expertise: ["Oracle", "PL/SQL", "MongoDB"],
    DOB: ISODate("1994-05-30"),
    EmailId: "sneha.iyer@oracle.com",
    Contact: "9876543218"
  },
  {
    Empid: 110,
    Name: { FName: "Vikas", LName: "Singh" },
    CompanyName: "Google",
    Salary: 120000,
    Designation: "Product Engineer",
    Age: 29,
    Expertise: ["Go", "Microservices", "Cloud"],
    DOB: ISODate("1996-07-14"),
```

```
    EmailId: "vikas.singh@google.com",
    Contact: "9876543219"
  }
])
```

4. Display the Document from Employee Collection

```
db.Employee.find().pretty()
```

5. Select all documents where the Designation field has the value "Software Engineer" and the value of the Salary is greater than 60000. (AND Operator)

```
db.Employee.find( {
      Designation: "Software Engineer",
      Salary: {$gt: 60000} } )
```

Output:

```
{
  _id: ObjectId('6972fb653c42eab6bccebea4'),
  Empid: 101,
  Name: { FName: 'Amit', LName: 'Sharma' },
  CompanyName: 'TCS',
  Salary: 65000,
  Designation: 'Software Engineer',
  Age: 25,
  Expertise: [ 'Java', 'Spring', 'SQL' ],
  DOB: ISODate('1999-02-15T00:00:00.000Z'),
  EmailId: 'amit.sharma@tcs.com',
  Contact: '9876543210'
}
```

6. Selects all documents in the collection where the field age has a value less than 27 or the value of the Salary field is greater than 90000. (OR Operator)

```
db.Employee.find(
  {
    $or: [
      { Age: { $lt: 27 } },
      { Salary: { $gt: 90000 } }
    ]
  }
)
```

Output:

```
{
  _id: ObjectId('6972fb653c42eab6bccebea4'),
  Empid: 101,
  Name: { FName: 'Amit', LName: 'Sharma' },
  CompanyName: 'TCS',
  Salary: 65000,
  Designation: 'Software Engineer',
  Age: 25,
  Expertise: [ 'Java', 'Spring', 'SQL' ],
  DOB: ISODate('1999-02-15T00:00:00.000Z'),
  EmailId: 'amit.sharma@tcs.com',
```

```
  Contact: '9876543210'
},
{
  _id: ObjectId('6972fb653c42eab6bccebea6'),
  Empid: 103,
  Name: { FName: 'Rahul', LName: 'Verma' },
  CompanyName: 'Wipro',
  Salary: 58000,
  Designation: 'Developer',
  Age: 24,
  Expertise: [ 'C++', 'DSA' ],
  DOB: ISODate('2000-01-05T00:00:00.000Z'),
  EmailId: 'rahul.verma@wipro.com',
  Contact: '9876543212'
},
{
  _id: ObjectId('6972fb653c42eab6bccebea7'),
  Empid: 104,
  Name: { FName: 'Pooja', LName: 'Kulkarni' },
  CompanyName: 'Capgemini',
  Salary: 69000,
  Designation: 'Backend Developer',
  Age: 26,
  Expertise: [ 'NodeJS', 'Express', 'MongoDB' ],
  DOB: ISODate('1999-08-20T00:00:00.000Z'),
  EmailId: 'pooja.k@capgemini.com',
  Contact: '9876543213'
},
{
  _id: ObjectId('6972fb653c42eab6bccebeaa'),
  Empid: 107,
  Name: { FName: 'Kiran', LName: 'Joshi' },
  CompanyName: 'HCL',
  Salary: 62000,
  Designation: 'Support Engineer',
  Age: 26,
  Expertise: [ 'Linux', 'Networking' ],
  DOB: ISODate('1998-12-18T00:00:00.000Z'),
  EmailId: 'kiran.joshi@hcl.com',
  Contact: '9876543216'
},
{
  _id: ObjectId('6972fb653c42eab6bccebead'),
  Empid: 110,
  Name: { FName: 'Vikas', LName: 'Singh' },
  CompanyName: 'Google',
  Salary: 120000,
  Designation: 'Product Engineer',
  Age: 29,
  Expertise: [ 'Go', 'Microservices', 'Cloud' ],
  DOB: ISODate('1996-07-14T00:00:00.000Z'),
  EmailId: 'vikas.singh@google.com',
  Contact: '9876543219'
```

```
}
```

7. Finds all documents with CompanyName: "TCS" and modifies their Salary field by 100000.

```
db.Employee.updateMany(
        { CompanyName: "TCS" },
        { $inc: { Salary: 100000 } }
  )

db.Employee.find({ CompanyName: "TCS" }).pretty()
```

Output:

```
{
    _id: ObjectId('6972fb653c42eab6bccebea4'),
    Empid: 101,
    Name: { FName: 'Amit', LName: 'Sharma' },
    CompanyName: 'TCS',
    Salary: 165000,
    Designation: 'Software Engineer',
    Age: 25,
    Expertise: [ 'Java', 'Spring', 'SQL' ],
    DOB: ISODate('1999-02-15T00:00:00.000Z'),
    EmailId: 'amit.sharma@tcs.com',
    Contact: '9876543210'
}
```

8. Find documents where Designation is equal to "Developer" only display _id, Empid, first name and last name.

```
db.Employee.find( { Designation: "Developer" },
 { _id: 1, Empid: 1, "Name.FName": 1, "Name.LName": 1 } )
```

Output:

```
{
    _id: ObjectId('6972fb653c42eab6bccebea6'),
    Empid: 103,
    Name: { FName: 'Rahul', LName: 'Verma' }
}
```

9. Find documents where Designation is not equal to "Developer" only display _id, Empid, first name and last name. (Not Equal Operator)

```
db.Employee.find(
       { Designation: { $ne: "Developer" } },
      { _id: 1, Empid: 1, "Name.FName": 1, "Name.LName": 1 }
      )
```

Output:

```
{
    _id: ObjectId('6972fb653c42eab6bccebea4'),
    Empid: 101,
    Name: { FName: 'Amit', LName: 'Sharma' }
},
{
```

```
    _id: ObjectId('6972fb653c42eab6bccebea5'),
    Empid: 102,
    Name: { FName: 'Neha', LName: 'Patil' }
  },
  {
    _id: ObjectId('6972fb653c42eab6bccebea7'),
    Empid: 104,
    Name: { FName: 'Pooja', LName: 'Kulkarni' }
  },
  {
    _id: ObjectId('6972fb653c42eab6bccebea8'),
    Empid: 105,
    Name: { FName: 'Suresh', LName: 'Rao' }
  },
  {
    _id: ObjectId('6972fb653c42eab6bccebea9'),
    Empid: 106,
    Name: { FName: 'Anjali', LName: 'Mehta' }
  },
  {
    _id: ObjectId('6972fb653c42eab6bccebeaa'),
    Empid: 107,
    Name: { FName: 'Kiran', LName: 'Joshi' }
  },
  {
    _id: ObjectId('6972fb653c42eab6bccebeab'),
    Empid: 108,
    Name: { FName: 'Rohit', LName: 'Deshmukh' }
  },
  {
    _id: ObjectId('6972fb653c42eab6bccebeac'),
    Empid: 109,
    Name: { FName: 'Sneha', LName: 'Iyer' }
  },
  {
    _id: ObjectId('6972fb653c42eab6bccebead'),
    Empid: 110,
    Name: { FName: 'Vikas', LName: 'Singh' }
  }
```

10. Find _id, Designation, and Name from all documents where Company_name is "Infosys".

```
Empdb> db.Employee.find( { CompanyName: "Infosys" },
          { _id: 1, Designation: 1, Name: 1 } )
```

Output:

```
  {
    _id: ObjectId('6972fb653c42eab6bccebea5'),
    Name: { FName: 'Neha', LName: 'Patil' },
    Designation: 'System Analyst'
  }
]
```

11. Selects all documents in the employee collection where the value of the Designation is either "Developer" or "Tester". **(IN Operator)**

```
db.Employee.find(
   {},
   { Designation: 1, Empid: 1, _id: 0 }
)
```

Output:

```
{ Empid: 101, Designation: 'Software Engineer' },
{ Empid: 102, Designation: 'System Analyst' },
{ Empid: 103, Designation: 'Developer' },
{ Empid: 104, Designation: 'Backend Developer' },
{ Empid: 105, Designation: 'Senior Engineer' },
{ Empid: 106, Designation: 'Consultant' },
{ Empid: 107, Designation: 'Support Engineer' },
{ Empid: 108, Designation: 'Full Stack Developer' },
{ Empid: 109, Designation: 'Database Admin' },
{ Empid: 110, Designation: 'Product Engineer' }
```

```
db.Employee.find(
   { Designation: { $in: ["Developer", "Tester"] } }
).pretty()
```

Output:

```
{
  _id: ObjectId('6972fb653c42eab6bccebea6'),
  Empid: 103,
  Name: { FName: 'Rahul', LName: 'Verma' },
  CompanyName: 'Wipro',
  Salary: 58000,
  Designation: 'Developer',
  Age: 24,
  Expertise: [ 'C++', 'DSA' ],
  DOB: ISODate('2000-01-05T00:00:00.000Z'),
  EmailId: 'rahul.verma@wipro.com',
  Contact: '9876543212'
}
```

12. Drop Single documents where age is less than 30. **(Less than Operator)**

```
db.Employee.find(
   {},
   {Age: 1, Empid: 1, _id: 0 }
)
```

Output:

```
[
 { Empid: 101, Age: 25 },
 { Empid: 102, Age: 27 },
 { Empid: 103, Age: 24 },
 { Empid: 104, Age: 26 },
 { Empid: 105, Age: 30 },
 { Empid: 106, Age: 28 },
```

```
{ Empid: 107, Age: 26 },
```

```
  { Empid: 108, Age: 27 },
  { Empid: 109, Age: 31 },
  { Empid: 110, Age: 29 }
]

db.Employee.deleteOne(
 { Age: { $lt: 30 } }
)
 { acknowledged: true, deletedCount: 1 }
```

Output:

```
Empdb> db.Employee.find( {}, { Age: 1, Empid: 1, _id: 0 } )
[
  { Empid: 102, Age: 27 },
  { Empid: 103, Age: 24 },
  { Empid: 104, Age: 26 },
  { Empid: 105, Age: 30 },
  { Empid: 106, Age: 28 },
  { Empid: 107, Age: 26 },
  { Empid: 108, Age: 27 },
  { Empid: 109, Age: 31 },
  { Empid: 110, Age: 29 }
]
```

13. Find employees who are NOT Developers and NOT earning more than 60000. (Nor Operator)

```
db.Employee.find(
{
     $nor: [
          { Designation: "Developer" },
          { Salary: { $gt: 60000 } }
          ]
     }
)
```

14. Find employees whose salary is NOT greater than 60000 (NOT Opeartor)

```
db.Employee.find(
  {
    Salary: { $not: { $gt: 60000 } }

  }
)
```

Assignment No 10

Statement: Aggregation and Indexing in MongoDB Design and execute MongoDB queries using aggregation and indexing techniques with suitable examples.

1. **Create Empdb Mangodb database**
2. **Create Employee collection by considering following Fields:**
   k) Empid: Number
   l) Name: Embedded Doc (FName, LName)
   m) Company Name: String
   n) Salary: Number
   o) Designation: String
   p) Age: Number
   q) Expertise: Array
   r) DOB: String or Date
   s) Email id: String
   t) Contact: String
3. **Frame suitable statements considering following MongoDB aggregation pipeline stages, expression operator.**

Pipeline stages

Stage	Purpose
$match	Filters documents
$project	Selects / reshapes fields
$group	Groups documents & aggregates data
$sort	Sorts documents
$limit	Limits number of documents
$skip	Skips documents
$count	Counts documents
$out	Stores result in new collection
$lookup	Performs join
$unwind	Deconstructs array field

Arithmetic Operators

Operator	Description
$sum	Total value
$avg	Average value
$min	Minimum value
$max	Maximum value

Comparison Operators

Operator	Description
$eq	Equal
$ne	Not equal
$gt	Greater than
$lt	Less than
$gte	Greater than or equal
$lte	Less than or equal

1. **Return Designation with Total Salary is Above 80000**

```
db.Employee.find(
  {},
   { Empid: 1, Salary: 1, Designation: 1, _id: 0 }
 )
```

Output:

```
[
  { Empid: 102, Salary: 72000, Designation: 'System Analyst'
},
  { Empid: 103, Salary: 58000, Designation: 'Developer' },
  { Empid: 104, Salary: 69000, Designation: 'Backend
Developer' },
  { Empid: 105, Salary: 85000, Designation: 'Senior
Engineer' },
  { Empid: 106, Salary: 76000, Designation: 'Consultant' },
  { Empid: 107, Salary: 62000, Designation: 'Support
Engineer' },
  { Empid: 108, Salary: 70000, Designation: 'Full Stack
Developer' },
  { Empid: 109, Salary: 90000, Designation: 'Database Admin'
},
  { Empid: 110, Salary: 120000, Designation: 'Product
Engineer' }
]
```

Aggregate Query:

```
db.Employee.aggregate([
    {
      $group: {
        _id: "$Designation",
        TotalSalary: { $sum: "$Salary" }
      }
    },
    {
      $match: {
        TotalSalary: { $gt: 80000 }
      }
    }
  ])
```

Output:

```
[
  { _id: 'Database Admin', TotalSalary: 90000 },
  { _id: 'Senior Engineer', TotalSalary: 85000 },
  { _id: 'Product Engineer', TotalSalary: 120000 }
]
```

2. Find total salary of employees whose salary is greater than 60000, grouped by company

```
db.Employee.aggregate([
  {
    $match: { Salary: { $gt: 60000 } }
  },
  {
    $group: {
      _id: "$CompanyName",
      TotalSalary: { $sum: "$Salary" }
    }
  }
```

```
])
```

Output:

```
[
 { _id: 'Oracle', TotalSalary: 90000 },
 { _id: 'TCS', TotalSalary: 349000 },
 { _id: 'IBM', TotalSalary: 85000 },
 { _id: 'Google', TotalSalary: 120000 }
]
```

3. Return Max and Min Salary for each company

```
db.Employee.find( {},
      { Empid: 1, Salary: 1, CompanyName: 1, _id: 0 } )
```

```
[
  { Empid: 102, CompanyName: 'TCS', Salary: 72000 },
  { Empid: 103, CompanyName: 'TCS', Salary: 58000 },
  { Empid: 104, CompanyName: 'TCS', Salary: 69000 },
  { Empid: 105, CompanyName: 'IBM', Salary: 85000 },
  { Empid: 106, CompanyName: 'TCS', Salary: 76000 },
  { Empid: 107, CompanyName: 'TCS', Salary: 62000 },
  { Empid: 108, CompanyName: 'TCS', Salary: 70000 },
  { Empid: 109, CompanyName: 'Oracle', Salary: 90000 },
  { Empid: 110, CompanyName: 'Google', Salary: 120000 }
]
```

```
db.Employee.aggregate([
  {
    $group: {
                _id: "$CompanyName",
                 MaxSalary: { $max: "$Salary" },
                  MinSalary: { $min: "$Salary" }
 } }
])
```

Output:

```
[

  { _id: 'Oracle', MaxSalary: 90000, MinSalary: 90000 },
  { _id: 'TCS', MaxSalary: 76000, MinSalary: 58000 },
  { _id: 'IBM', MaxSalary: 85000, MinSalary: 85000 },
  { _id: 'Google', MaxSalary: 120000, MinSalary: 120000 }
]
```

4. Returns names and _id in upper case and in alphabetical order.

```
db.Employee.find(
        {},
       { _id: 1, Name: 1 }
)
```

Ouput:

```
[
 {
   _id: ObjectId('6972fb653c42eab6bccebea5'),
   Name: { FName: 'Neha', LName: 'Patil' }
 },
```

```
  {
    _id: ObjectId('6972fb653c42eab6bccebea6'),
    Name: { FName: 'Rahul', LName: 'Verma' }
  },
  {
    _id: ObjectId('6972fb653c42eab6bccebea7'),
    Name: { FName: 'Pooja', LName: 'Kulkarni' }
  },
  {
    _id: ObjectId('6972fb653c42eab6bccebea8'),
    Name: { FName: 'Suresh', LName: 'Rao' }
  },
  {
    _id: ObjectId('6972fb653c42eab6bccebea9'),
    Name: { FName: 'Anjali', LName: 'Mehta' }
  },
  {
    _id: ObjectId('6972fb653c42eab6bccebeaa'),
    Name: { FName: 'Kiran', LName: 'Joshi' }
  },
  {
    _id: ObjectId('6972fb653c42eab6bccebeab'),
    Name: { FName: 'Rohit', LName: 'Deshmukh' }
  },
  {
    _id: ObjectId('6972fb653c42eab6bccebeac'),
    Name: { FName: 'Sneha', LName: 'Iyer' }
  },
  {
    _id: ObjectId('6972fb653c42eab6bccebead'),
    Name: { FName: 'Vikas', LName: 'Singh' }
  }
```

Aggregate Method

```
db.Employee.aggregate([
  {
    $project: {
      _id: 1,
      FullName: { $concat: [ { $toUpper: "$Name.FName" }, " ",
     { $toUpper: "$Name.LName" } ] }
    }
  },
  {
    $sort: { FullName: 1 }  // Sort alphabetically
  }
])
```

Output:

```
[
  {
    _id: ObjectId('6972fb653c42eab6bccebea9'),
    FullName: 'ANJALI MEHTA'
  },
```

```
  {
    _id: ObjectId('6972fb653c42eab6bccebeaa'),
    FullName: 'KIRAN JOSHI'
  },
  { _id: ObjectId('6972fb653c42eab6bccebea5'),
          FullName: 'NEHA PATIL' },
  {
    _id: ObjectId('6972fb653c42eab6bccebea7'),
    FullName: 'POOJA KULKARNI'
  },
  {
    _id: ObjectId('6972fb653c42eab6bccebea6'),
    FullName: 'RAHUL VERMA'
  },
  {
    _id: ObjectId('6972fb653c42eab6bccebeab'),
    FullName: 'ROHIT DESHMUKH'
  },
  { _id: ObjectId('6972fb653c42eab6bccebeac'),
        FullName: 'SNEHA IYER' },
  { _id: ObjectId('6972fb653c42eab6bccebea8'),
         FullName: 'SURESH RAO' },
  {
    _id: ObjectId('6972fb653c42eab6bccebead'),
    FullName: 'VIKAS SINGH'
  }
]
```

5. Calculate Total Salary by Company name and store output to new collection

```
db.Employee.find( {},
 { Empid: 1, Salary: 1, CompanyName: 1, _id: 0 } )
[
  { Empid: 102, CompanyName: 'TCS', Salary: 72000 },
  { Empid: 103, CompanyName: 'TCS', Salary: 58000 },
  { Empid: 104, CompanyName: 'TCS', Salary: 69000 },
  { Empid: 105, CompanyName: 'IBM', Salary: 85000 },
  { Empid: 106, CompanyName: 'TCS', Salary: 76000 },
  { Empid: 107, CompanyName: 'TCS', Salary: 62000 },
  { Empid: 108, CompanyName: 'TCS', Salary: 70000 },
  { Empid: 109, CompanyName: 'Oracle', Salary: 90000 },
  { Empid: 110, CompanyName: 'Google', Salary: 120000 }
]
db.Employee.aggregate([
  {
    $group: {
      _id: "$CompanyName",
      TotalSalary: { $sum: "$Salary" }
    }
  },
  {
    $out: "TotalSalaryByCompany"
  }
])
```

Output:

```
db.TotalSalaryByCompany.find().pretty()
[
 { _id: 'Oracle', TotalSalary: 90000 },
 { _id: 'TCS', TotalSalary: 407000 },
 { _id: 'IBM', TotalSalary: 85000 },
 { _id: 'Google', TotalSalary: 120000 }
]
```

6. Count of Employees per Designation

```
db.Employee.aggregate([
  {
    $group: {
      _id: "$Designation",
      Count: { $sum: 1 }
    }
  }
])
```

Output:

```
[
  { _id: 'Database Admin', Count: 1 },
  { _id: 'Support Engineer', Count: 1 },
  { _id: 'Developer', Count: 1 },
  { _id: 'System Analyst', Count: 1 },
  { _id: 'Backend Developer', Count: 1 },
  { _id: 'Full Stack Developer', Count: 1 },
  { _id: 'Consultant', Count: 1 },
  { _id: 'Senior Engineer', Count: 1 },
  { _id: 'Product Engineer', Count: 1 }
]
```

7. Calculate the average salary of each company from the Employee collection

```
db.Employee.aggregate([
  {
        $group: {
        _id: "$CompanyName",
  AverageSalary: { $avg: "$Salary" }
  }
  }
  ])
```

Output:

```
[

{ "_id": "TCS", "AverageSalary": 65000 },
{ "_id": "Infosys", "AverageSalary": 72000 },
{ "_id": "Wipro", "AverageSalary": 58000 },
{ "_id": "Capgemini", "AverageSalary": 69000 },
{ "_id": "IBM", "AverageSalary": 85000 },
{ "_id": "Accenture", "AverageSalary": 76000 },
{ "_id": "HCL", "AverageSalary": 62000 },
{ "_id": "Tech Mahindra", "AverageSalary": 70000 },
{ "_id": "Oracle", "AverageSalary": 90000 },
```

```
{ "_id": "Google", "AverageSalary": 120000 }
]
```

8. Count employees whose salary is greater than 70000

```
db.Employee.aggregate([
{ $match: { Salary: { $gt: 70000 } } },
{ $count: "TotalEmployees" }
])
```

Output:

```
[
{ "TotalEmployees": 5 }
]
```

9. Display the total count of Employee

```
db.Employee.aggregate([
  {
    $count: "TotalEmployees"
  }
])
```

Output:

```
[ { TotalEmployees: 9 } ]
```

10. Insert 200000 in user collection and create index on username. Observe the differences between in output before and after Index.

```
var users = [];

for (i = 0; i < 200000; i++) {
  users.push({
    i: i,
    username: "user" + i,
    age: Math.floor(Math.random() * 120),
    created: new Date()
  });
}

db.user.insertMany(users);
```

db.user.count()

1. Before Index Creation : Find all documents in collection

```
db.user.find({}).explain("executionStats")
```

```
executionStats: {
    executionSuccess: true,
    nReturned: 200000,
    executionTimeMillis: 172,
    totalKeysExamined: 0,
    totalDocsExamined: 200000,
    executionStages: {
      isCached: false,
      stage: 'COLLSCAN',
```

```
            nReturned: 200000,
            executionTimeMillisEstimate: 29,
```

2. Before Index Creation: Find user0 in collection

```
db.user.find({ username: "user0"}).explain("executionStats")

     executionStats: {
         executionSuccess: true,
         nReturned: 1,
         executionTimeMillis: 123,
         totalKeysExamined: 0,
         totalDocsExamined: 200000,
         executionStages: {
           isCached: false,
           stage: 'COLLSCAN',
           filter: { username: { '$eq': 'user0' } },
           nReturned: 1,
           executionTimeMillisEstimate: 117,
```

3. Before Index Creation: Find user19999 in collection

```
db.user.find({ username: "user19999" }).explain("executionStats")

    executionStats: {
        executionSuccess: true,
        nReturned: 1,
        executionTimeMillis: 122,
        totalKeysExamined: 0,
        totalDocsExamined: 200000,
        executionStages: {
          isCached: false,
          stage: 'COLLSCAN',
          filter: { username: { '$eq': 'user19999' } },
          nReturned: 1,
          executionTimeMillisEstimate: 107,
```

4. Before Index Creation: Find user9999 in collection

```
db.user.find({ username: "user9999" }).explain("executionStats")

       executionStats: {
         executionSuccess: true,
         nReturned: 1,
         executionTimeMillis: 126,
         totalKeysExamined: 0,
         totalDocsExamined: 200000,
         executionStages: {
           isCached: false,
           stage: 'COLLSCAN',
           filter: { username: { '$eq': 'user9999' } },
           nReturned: 1,
           executionTimeMillisEstimate: 102,
```

==========

To create Single Field Index on Username

```
db.user.createIndex({ username: 1 })
```

1. After Index Creation : Find all documents in collection

```
db.user.find({}).explain("executionStats")
```

```
executionStats: {
executionSuccess: true,
nReturned: 200000,
executionTimeMillis: 82,
totalKeysExamined: 0,
totalDocsExamined: 200000,
executionStages: {
  isCached: false,
  stage: 'COLLSCAN',
  nReturned: 200000,
  executionTimeMillisEstimate: 39,
```

2. After Index Creation: Find user0 in collection

```
db.user.find({ username: "user0"}).explain("executionStats")
```

```
executionStats: {
executionSuccess: true,
nReturned: 1,
executionTimeMillis: 54,
totalKeysExamined: 1,
totalDocsExamined: 1,
executionStages: {
  isCached: false,
  stage: 'FETCH',
  nReturned: 1,
  executionTimeMillisEstimate: 0,
```

3. After Index Creation: Find user19999 in collection

```
db.user.find({ username: "user19999" }).explain("executionStats")
```

```
executionStats: {
executionSuccess: true,
nReturned: 1,
executionTimeMillis: 0,
totalKeysExamined: 1,
totalDocsExamined: 1,
executionStages: {
  isCached: false,
```

```
          stage: 'FETCH',
          nReturned: 1,
          executionTimeMillisEstimate: 0,
```

4.After Index Creation: Find user9999 in collection

```
db.user.find({ username: "user9999" }).explain("executionStats")
```

```
      executionStats: {
         executionSuccess: true,
         nReturned: 1,
         executionTimeMillis: 0,
         totalKeysExamined: 1,
         totalDocsExamined: 1,
         executionStages: {
           isCached: false,
           stage: 'FETCH',
           nReturned: 1,
           executionTimeMillisEstimate: 0,
```

***** THE END ****